UNIVERSIDAD CONTINENTAL
ESCUELA DE POSGRADO
MAESTRÍA EN CIENCIA DE DATOS

Gestión de datos y bases de datos con SQL (PSMA00611)

Base de datos para una aerolínea comercial en Estados Unidos

Avance del Producto Académico Colaborativo

Integrantes:

Carlos Veliz de Villa Aguirre

Elias Sifuentes Valles

Carlos De los Ríos De la Roca (líder del equipo)

Docentes: Walter Alejandro Silva Sotillo y Max Leoncio Mucha Morales

Sección / NRC: [POR COMPLETAR]

Repositorio: https://github.com/CARLOSDELOSRIOS/aerolinea-bd-psma00611

Lugar y fecha de entrega: [POR COMPLETAR], 2026

Índice

Resumen **4**

Qué nos pide el curso **5**

El encargo en un esquema 5

Dónde estamos 5

Nuestro equipo y cómo trabajamos **7**

Quiénes somos 7

Cómo trabajamos 7

Lo que muestra GitHub 8

El problema y lo que queremos lograr **9**

El problema 9

Nuestros objetivos 9

Cómo entendimos los requisitos **10**

Las diez relaciones del enunciado 10

Pasajero, comprador y cliente no son lo mismo 10

De la ruta al vuelo con fecha 11

El precio de un tiquete no cambia después de la venta 11

Otras decisiones 12

Cómo diseñamos el modelo de datos **13**

El modelo conceptual 13

Normalización 13

Qué hemos construido **15**

Los scripts y su orden 15

Las reglas viven dentro de la base 15

La aplicación en Python 16

Arquitectura local y en la nube 17

Qué hemos comprobado **19**

Las pruebas 19

Ventas simultáneas sin sobreventa 19

Primeros resultados 20

Lo que aprendimos en el camino 21

Lo que sigue **22**

Plan de trabajo 22

Riesgos 22

Preguntas para el docente 23

Conclusiones del avance **24**

Referencias **25**

Anexo A. Árbol de archivos del repositorio **26**

Anexo B. Prompts utilizados **31**

Anexo C. Declaración sobre el uso de inteligencia artificial **40**

Índice de figuras

Figura 1. Qué pide el curso: los tres momentos de evaluación y los seis componentes del producto.....5
Figura 2. Estado de cada componente del encargo a la fecha del avance.....6
Figura 3. Cómo trabajamos los tres: del prompt al repositorio.....7
Figura 4. Objetivo general, objetivos específicos y su estado.....9
Figura 5. Persona, comprador, pasajero y cliente registrado, con un ejemplo.....11
Figura 6. De la ruta al vuelo con fecha, con un ejemplo real de los datos de referencia.....11
Figura 7. Versiones de la tarifa JFK → LAX en económica y precio que conserva un tiquete comprado durante la versión 1.....12
Figura 8. Modelo conceptual: 19 entidades y sus relaciones.....13
Figura 9. Normalización paso a paso, desde una planilla de ventas hasta la tercera forma normal.....14
Figura 10. Orden de ejecución de los scripts T-SQL.....15
Figura 11. Capas de integridad: lo que comprueba la base antes de guardar un dato.....16
Figura 12. Comandos de la aplicación en Python y su secuencia.....17
Figura 13. Arquitectura de la solución: laptop con SQL Server local y Azure SQL Database.....18
Figura 14. Pruebas superadas y entornos pendientes.....19
Figura 15. Prueba de concurrencia: 40 sesiones simultáneas y 12 asientos.....20
Figura 16. Ventas, cobros y reembolsos por mes (Q03).....21
Figura 17. Plan de trabajo propuesto hasta la entrega final.....22
Figura 18. Riesgos identificados y medidas previstas.....23
Figura 19. Carpetas del repositorio y número de archivos.....26

Índice de tablas

Tabla 1. Integrantes del equipo.....7
Tabla 2. Relaciones del enunciado y tablas que las implementan.....10
Tabla 3. Otras decisiones que tomamos sobre el enunciado.....12
Tabla 4. Indicadores preliminares con datos sintéticos.....20
Tabla 5. Plan de trabajo propuesto.....22
Tabla 6. Preguntas que queremos hacer en la retroalimentación del avance.....23

Resumen

Este documento cuenta en qué punto está nuestro Producto Académico Colaborativo: una base de datos en la nube para una aerolínea nueva que empezará a volar en Estados Unidos (Universidad Continental, 2026). Lo escribimos para la evaluación del avance, pero también para ordenar lo que ya hicimos y lo que nos falta antes de la entrega final del 16 de noviembre de 2026.

Para el avance, las guías del curso piden tres cosas: registrarse en GitHub, crear el equipo y el proyecto, y subir los scripts (Universidad Continental, 2023). El repositorio ya existe en https://github.com/CARLOSDELOSRIOS/aerolinea-bd-psma00611 y contiene los scripts T-SQL, la aplicación en Python, las pruebas, la documentación y los borradores del informe y de la presentación. Nos queda sumar como colaboradores a los otros dos integrantes, para que cada uno suba sus cambios con su propia cuenta.

En contenido vamos bien encaminados. Ya tenemos una primera versión completa de los seis componentes que pide el encargo: requisitos, modelo de datos, creación de tablas, carga de datos, consultas y consultas desde Python. La base tiene 39 tablas de negocio y hace cumplir 42 reglas por sí misma. La probamos en un entorno de desarrollo y superó todas las pruebas: 23 de reglas de negocio, 4 de concurrencia y 23 de la aplicación.

Lo que todavía falta, y lo decimos con claridad, es ejecutar el proyecto en nuestras laptops y desplegarlo en Azure SQL Database, que es el requisito de nube del encargo. El documento termina con nuestro plan para lograrlo, los riesgos que vemos y las preguntas que queremos hacer al docente.

Qué nos pide el curso

El encargo en un esquema

El curso evalúa el producto en tres momentos: el avance, la presentación oral y la entrega final, con pesos de 20 %, 40 % y 40 % (Universidad Continental, 2023). Las guías incluyen una rúbrica de cuatro criterios para el producto: información, análisis, conclusiones, y estilo y gramática. La figura 1 resume qué pide cada momento y qué debe contener el producto.

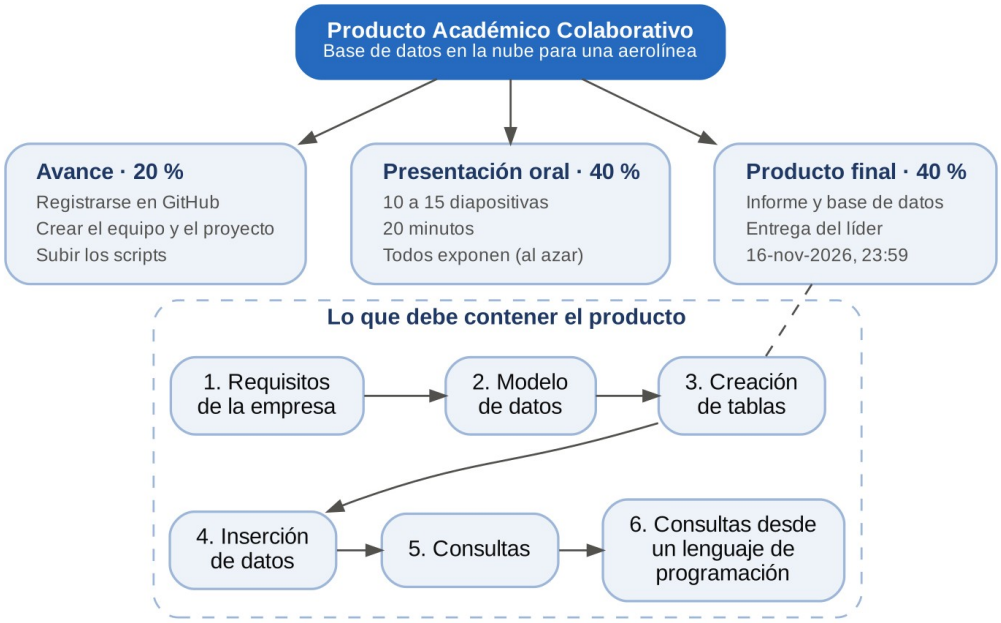


Figura 1. Qué pide el curso: los tres momentos de evaluación y los seis componentes del producto.
Fuente: guías de aprendizaje y enunciado del aula virtual.

El enunciado del aula virtual no menciona el avance; aparece en el sílabo y en las guías. Tampoco coinciden las fechas: según el documento, el avance va en la semana 2, 3 o 4. No dimos por hecho que la guía de los viernes sea la vigente; preparamos el avance con tiempo para llegar a cualquiera de las tres fechas y lo incluimos entre las preguntas al docente.

Dónde estamos

La figura 2 muestra el estado de cada componente. El verde indica lo que ya está hecho y probado; el ámbar, lo que avanza; y el rojo, lo que aún no empieza. Cada estado lleva además un símbolo y una palabra, para que se entienda también en blanco y negro.

Componente	Qué tenemos	Estado
Requisitos de la empresa	42 reglas, 13 supuestos, 13 casos de uso	✔ Completo
Modelo de datos	Conceptual, lógico y físico en 3FN; 269 columnas documentadas	✔ Completo
Creación de tablas	Scripts 01 a 13 probados en el entorno de desarrollo	✔ Completo
Inserción de datos	Catálogos y ≈1,19 millones de filas sintéticas	✔ Completo
Consultas	20 consultas con resultado e interpretación	✔ Completo
Consultas desde Python	pyodbc y pandas, con parámetros enlazados	✔ Completo
Repositorio en GitHub	Proyecto y scripts subidos; falta sumar colaboradores	◐ En curso
Ejecución en las laptops	Guía y script de verificación listos	○ Pendiente
Base de datos en Azure	Guía y scripts listos; sin desplegar	○ Pendiente

Figura 2. Estado de cada componente del encargo a la fecha del avance.
Fuente: repositorio del proyecto y evidencias del entorno de desarrollo.

Nuestro equipo y cómo trabajamos

Quiénes somos

Tabla 1. Integrantes del equipo

Integrante	Rol	Participación
Carlos Veliz de Villa Aguirre	Integrante	Todas las etapas
Elias Sifuentes Valles	Integrante	Todas las etapas
Carlos De los Ríos De la Roca	Líder: envía el producto en el aula virtual y administra el repositorio	Todas las etapas

No nos repartimos el trabajo por áreas. Los tres participamos en todas las etapas: decidimos juntos qué pedir en cada momento, redactamos y revisamos juntos los prompts, y revisamos juntos los resultados. Así lo declaramos como equipo.

Cómo trabajamos

Usamos Claude Code, un asistente de programación de Anthropic, como herramienta de apoyo. El ciclo de trabajo fue siempre el mismo (figura 3): redactamos el prompt, la herramienta genera los archivos y ejecuta las pruebas, revisamos el resultado y lo guardamos en GitHub. Cuando algo no nos convencía, ajustábamos el prompt y repetíamos el ciclo.

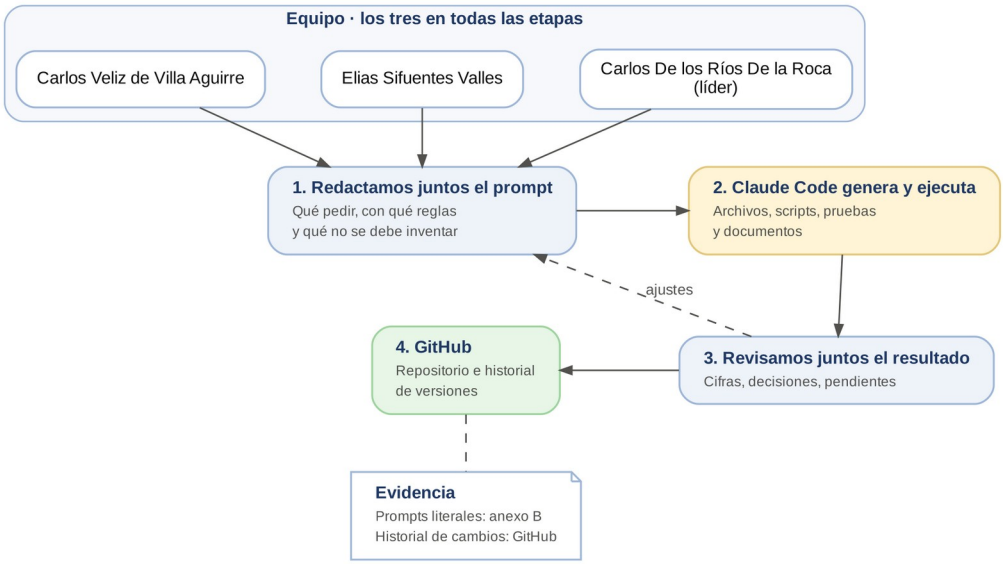


Figura 3. Cómo trabajamos los tres: del prompt al repositorio.
Fuente: declaración del equipo.

En los prompts dejamos reglas claras. Por ejemplo, no inventar integrantes, cifras, referencias ni pruebas; marcar como pendiente lo que no se pudo comprobar; y no afirmar que las pruebas locales

cumplen el requisito de nube. Por eso este documento distingue con cuidado lo hecho de lo pendiente. El docente nos pidió adjuntar los prompts: están completos y sin editar en el anexo B.

Lo que muestra GitHub

El repositorio pertenece a la cuenta de Carlos De los Ríos De la Roca. Los primeros commits aparecen con el autor «Claude», porque los cambios se generaron y se subieron desde la sesión de la herramienta. No representan el aporte de una sola persona, sino el trabajo conjunto descrito arriba. Desde ahora, cada integrante subirá sus cambios con su propia cuenta. Antes de entregar debemos comprobar en *Settings* → *Collaborators* que Carlos Veliz de Villa Aguirre y Elias Sifuentes Valles figuren como colaboradores.

El problema y lo que queremos lograr

El problema

Una aerolínea necesita saber con exactitud qué vuelos ofrece, quién compra y quién viaja, cuánto cobró y cuánto devolvió, y qué avión y qué tripulación opera cada vuelo. Un error en esos datos se nota enseguida: un asiento vendido dos veces, un vuelo con más pasajeros que asientos o un reembolso mal calculado. Además, la información debe servir para decidir —qué rutas se llenan, qué canales venden más—, y para eso todos tienen que calcular las cifras de la misma manera.

Nuestros objetivos

Nos propusimos un objetivo general y cinco objetivos específicos (figura 4). Cuatro ya tienen una versión completa y probada; el quinto avanza: las pruebas están hechas, pero falta el despliegue en la nube.

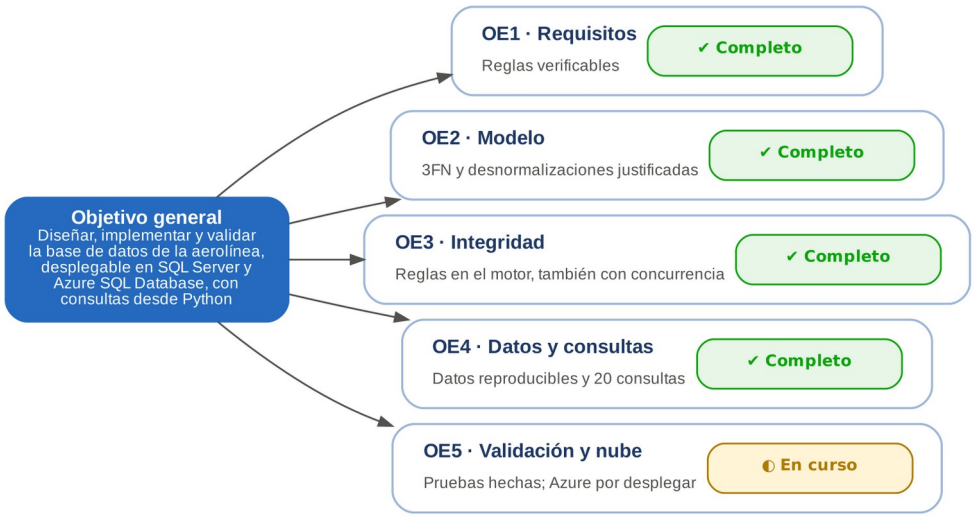


Figura 4. Objetivo general, objetivos específicos y su estado.
Fuente: elaboración del equipo.

Cómo entendimos los requisitos

Las diez relaciones del enunciado

El enunciado enumera diez relaciones que la base debe tener como mínimo y pide «ser exhaustivo con los atributos» (Universidad Continental, 2026). La tabla 2 muestra cómo las implementamos y cuántos atributos tiene cada tabla principal; el diccionario de datos del repositorio describe cada columna con su tipo, dominio y definición.

Tabla 2. Relaciones del enunciado y tablas que las implementan

Relación del enunciado	Tablas	Atributos
Clientes	`ven.persona` (toda persona que compra o viaja) y `ven.cliente` (persona con cuenta)	12 + 8
Empleados	`rh.empleado`, `rh.cargo`, `rh.departamento`	14
Rutas (origen, destino, día y hora)	`ops.ruta`, `ops.programacion`, `ops.programacion_dia`	7 + 9 + 2
Vuelos (generados de una ruta programada)	`ops.vuelo`, creado por `ops.usp_generar_vuelos`	14
Localizaciones (físicas y web)	`ref.localizacion` (aeropuerto, oficina o canal web) y `ref.aeropuerto`	14 + 7
Tiquetes (clientes registrados o no)	`ven.compra`, `ven.tiquete`	7 + 20
Aviones (asignados a viajes)	`ops.avion`, `ops.configuracion_asientos`	9
Cabina (VIP o económica; precio por ruta y cabina)	`ref.cabina`, `ops.vuelo_cabina`, `ven.tarifa`	7 + 4 + 7
Pagos (uno o más por tiquete)	`ven.pago`, `ven.reembolso`	12 + 8
Descuentos (porcentaje sobre el precio original)	`ven.descuento`	11

Pasajero, comprador y cliente no son lo mismo

La primera duda fue quién es «el cliente». El enunciado dice que también compran personas sin registro, y en la práctica quien paga no siempre es quien viaja. Por eso separamos tres ideas (figura 5): toda persona puede comprar o viajar, y solo algunas tienen una cuenta registrada. Así, una compra sin registro no deja campos vacíos ni duplica a nadie.

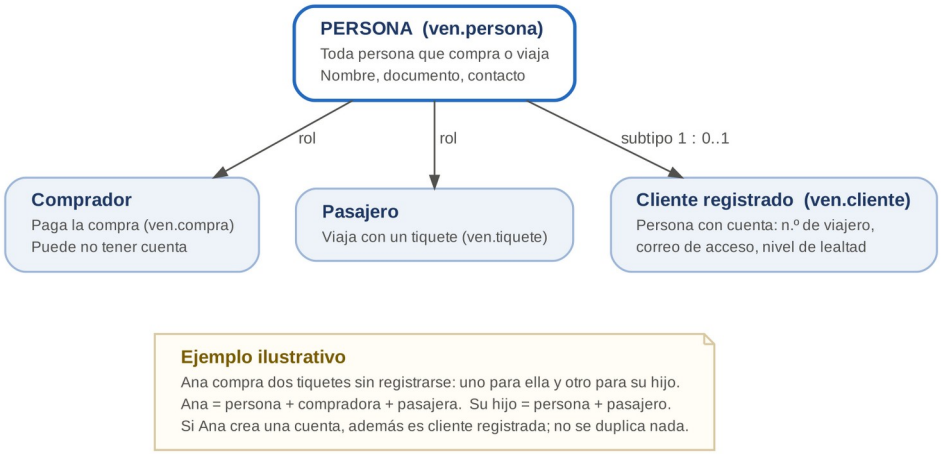


Figura 5. Persona, comprador, pasajero y cliente registrado, con un ejemplo.
Fuente: modelo de datos del proyecto.

De la ruta al vuelo con fecha

La segunda duda fue la diferencia entre ruta y vuelo. Lo resolvimos en tres niveles (figura 6): la ruta es solo el par origen-destino; la programación fija el número de vuelo, la hora y los días; y el vuelo es una salida con fecha, que es la que tiene avión, tripulación y estado. El procedimiento `ops.usp_generar_vuelos` crea los vuelos a partir de la programación, tal como pide el enunciado.

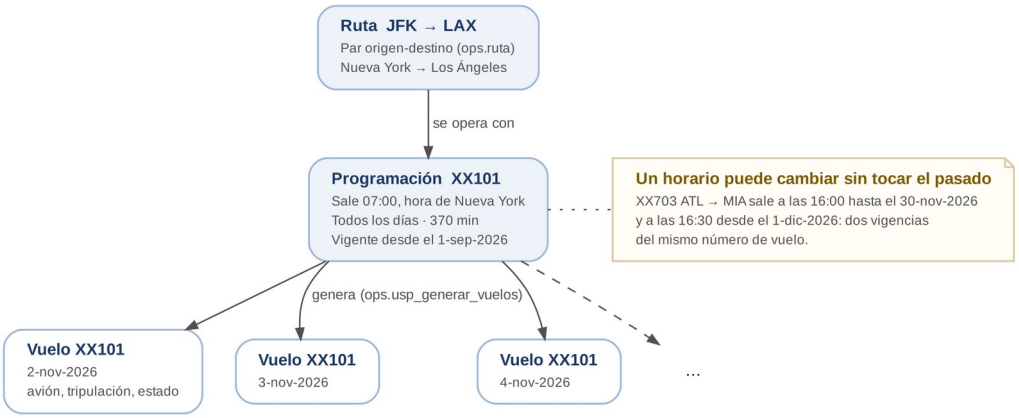


Figura 6. De la ruta al vuelo con fecha, con un ejemplo real de los datos de referencia.
Fuente: sql/09_datos_referencia.sql.

El precio de un tiquete no cambia después de la venta

Las tarifas cambian con el tiempo, pero un tiquete vendido debe conservar el precio pactado. Por eso el tiquete guarda una copia del precio y del descuento del momento de la venta, y una tarifa ya usada no se edita: se cierra y se crea otra versión. La figura 7 lo muestra con la tarifa real de la ruta JFK-LAX en económica.

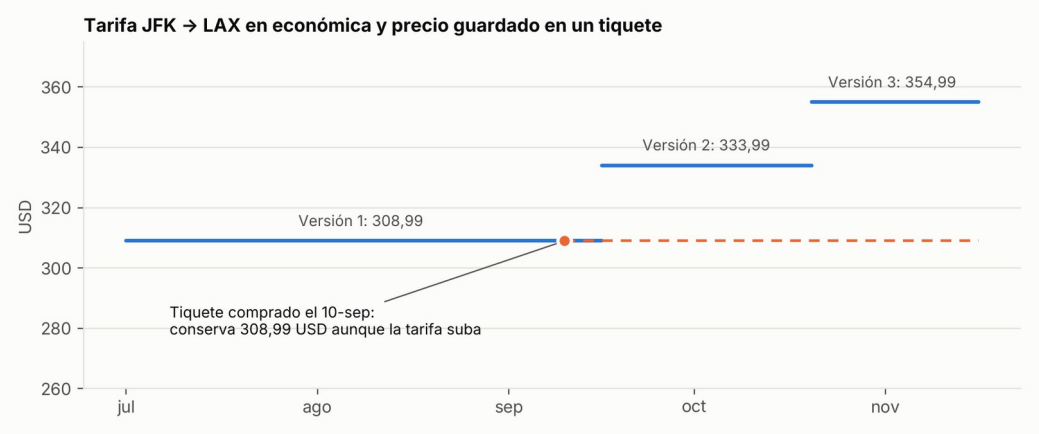


Figura 7. Versiones de la tarifa JFK → LAX en económica y precio que conserva un tiquete comprado durante la versión 1.

Fuente: sql/09_datos_referencia.sql y consulta Q06.

Otras decisiones

Tabla 3. Otras decisiones que tomamos sobre el enunciado

Pregunta	Lo que decidimos	Por qué
¿Se acumulan los descuentos?	Un solo descuento por tiquete: el mayor que corresponda	Evita promociones acumuladas sin control
¿Cómo se paga en partes?	Varios pagos por tiquete; se emite al completar el precio; la reserva vence a las 72 horas	El enunciado permite uno o más pagos por tiquete
¿Qué hora se guarda si la red cruza cinco husos horarios?	La hora local con su desfase en los vuelos y la hora UTC en las transacciones	Evita errores con el horario de verano y las llegadas al día siguiente
¿Cuándo se devuelve el dinero?	Cancelación sin penalidad en las primeras 24 horas si se compró con 7 días o más de anticipación; luego, según la cabina	Sigue la norma del Departamento de Transporte de Estados Unidos (Customer Service Plan, 2026)

Donde el enunciado no decía nada, lo completamos con supuestos explícitos: son trece (SUP-01 a SUP-13) y están en el análisis de requisitos del repositorio.

Cómo diseñamos el modelo de datos

El modelo conceptual

La figura 8 muestra las 19 entidades del modelo y sus relaciones. Todo gira alrededor del tiquete. La pieza que evita la sobreventa es el cupo por vuelo y cabina: cada venta ocupa una plaza de ese cupo, y la base no deja vender si no queda ninguna.

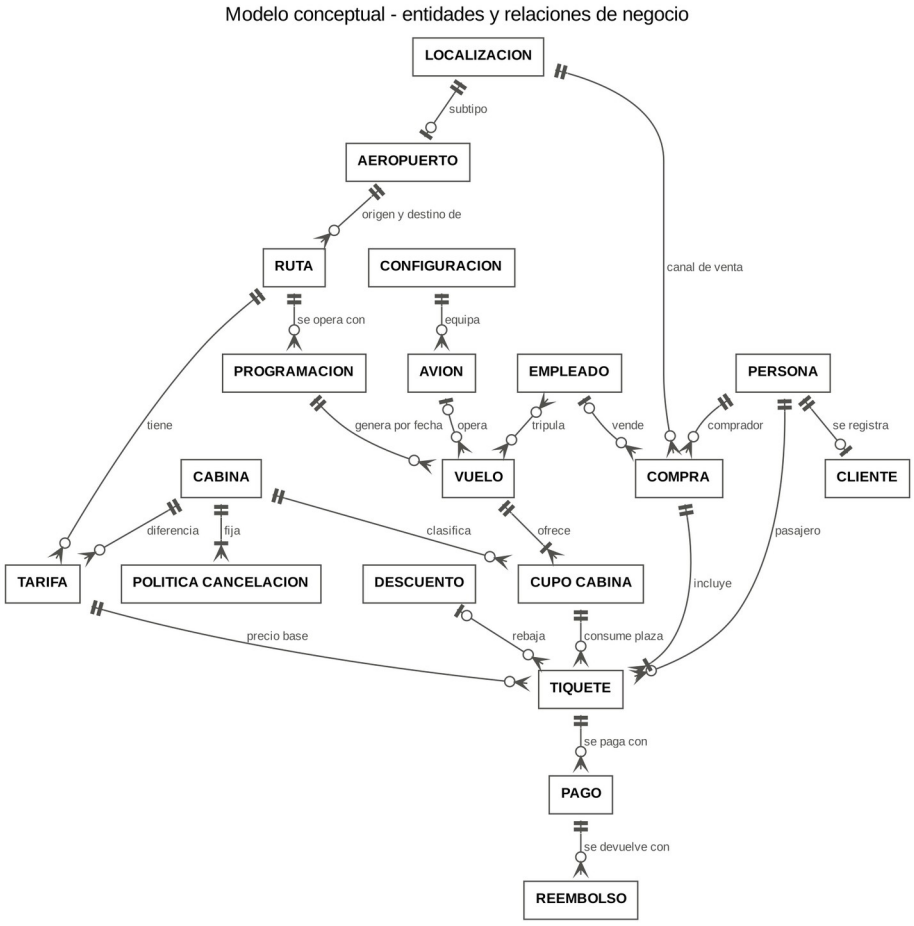


Figura 8. Modelo conceptual: 19 entidades y sus relaciones.
Fuente: docs/02_modelo/diagramas/conceptual.mmd.

A partir de este modelo construimos cuatro diagramas lógicos (red y vuelos; flota y tripulación; personas y tiquetes; precios y pagos) y el modelo físico. Los diagramas físicos y el diccionario se generan directamente desde la base, de modo que no pueden contradecir al código.

Normalización

Normalizar es repartir los datos en tablas de modo que cada dato se guarde una sola vez y en el lugar correcto (Codd, 1970; Elmasri y Navathe, 2011). Partimos de una planilla de ventas imaginaria con todo junto y la fuimos descomponiendo hasta la tercera forma normal (figura 9).

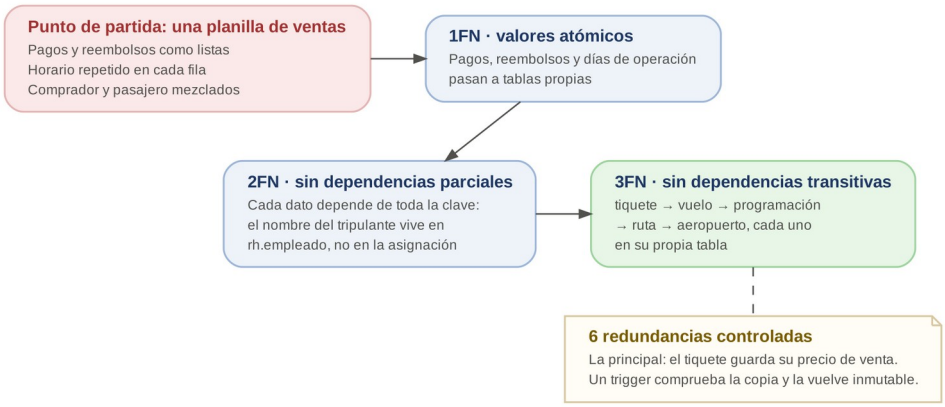


Figura 9. Normalización paso a paso, desde una planilla de ventas hasta la tercera forma normal.
Fuente: docs/02_modelo/normalizacion.md.

Mantuvimos seis redundancias a propósito, cada una con su justificación y su control. La más importante es la copia del precio en el tiquete. No genera inconsistencias porque el precio del tiquete es un hecho de la venta, no la tarifa de hoy: un trigger comprueba que la copia coincida con la tarifa vigente al vender e impide modificarla después.

Qué hemos construido

Los scripts y su orden

Organizamos los scripts T-SQL en el orden en que deben ejecutarse (figura 10). Todos se pueden volver a ejecutar sin errores ni duplicados. Lo único que borra la base está aislado en un script marcado como destructivo, que pide confirmación y se niega a funcionar en Azure.

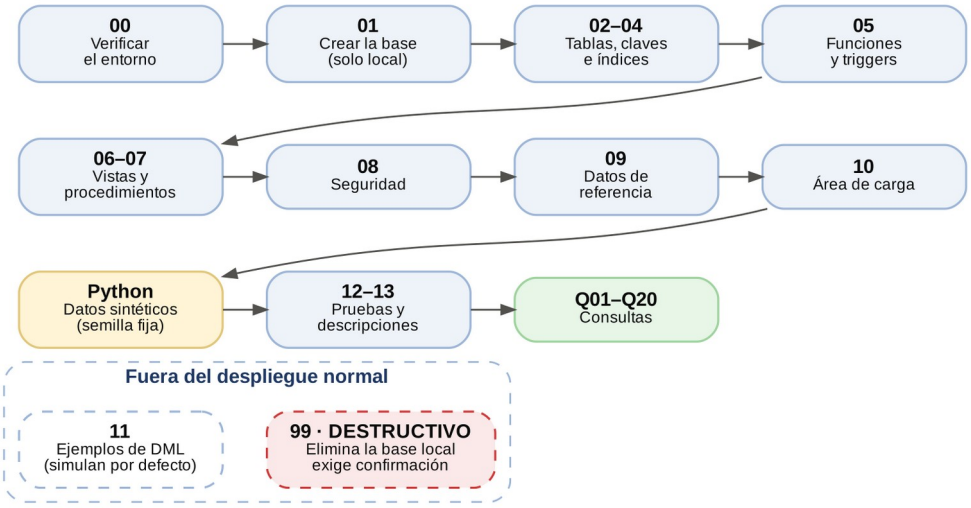


Figura 10. Orden de ejecución de los scripts T-SQL.
Fuente: sql/README.md.

Las reglas viven dentro de la base

Queríamos que la base hiciera cumplir sus reglas aunque alguien insertara datos sin pasar por la aplicación, por ejemplo desde SSMS. Para eso las reglas se organizan en capas (figura 11). Primero, restricciones declarativas, que son las más simples y seguras; después, triggers, solo donde una restricción no alcanza, como al validar un cambio de estado o impedir que un avión tenga dos vuelos a la vez; luego, procedimientos almacenados para las operaciones completas; y, al final, consultas de control.

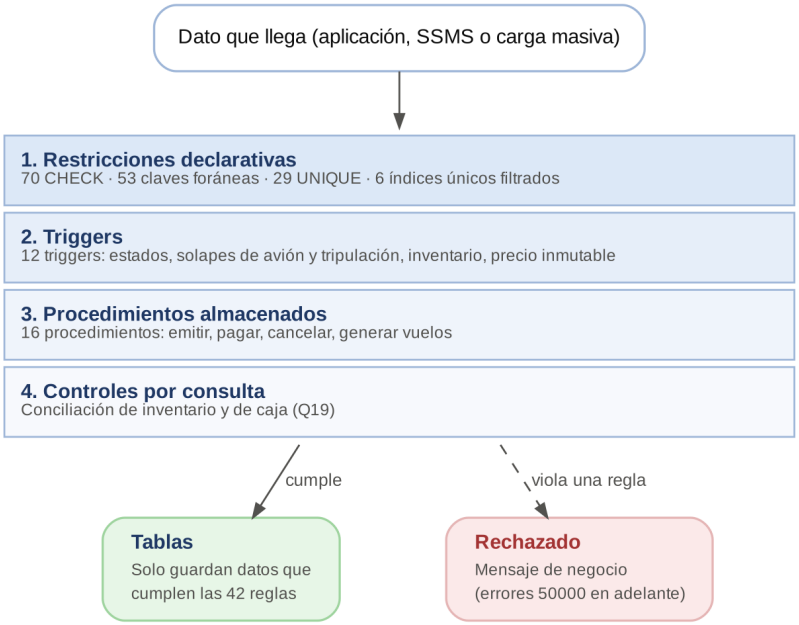


Figura 11. Capas de integridad: lo que comprueba la base antes de guardar un dato.
Fuente: catálogo de la base y docs/01_requisitos/analisis_requisitos.md.

La aplicación en Python

La aplicación de consola automatiza el trabajo repetitivo (figura 12): comprueba la conexión, despliega los scripts, genera y carga los datos sintéticos, ejecuta las pruebas y las consultas, y produce los CSV, los gráficos y los documentos. Las consultas reciben sus valores como parámetros, nunca pegados al texto SQL, para evitar la inyección de código. La configuración se lee de un archivo `.env` que no se sube a GitHub, y en las laptops usamos autenticación de Windows, así que no hay contraseñas guardadas.

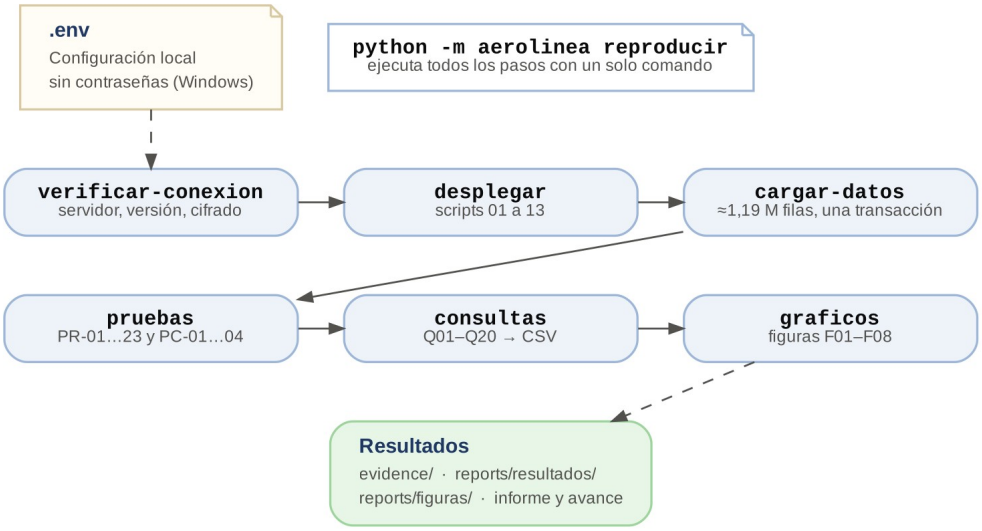


Figura 12. Comandos de la aplicación en Python y su secuencia.
Fuente: src/aerolinea/cli.py.

Arquitectura local y en la nube

El mismo código T-SQL está pensado para la laptop y para Azure SQL Database (figura 13). Las diferencias entre ambos entornos se resolvieron en el diseño: los scripts de despliegue no usan `USE`, la conexión siempre va cifrada y, en Azure, el certificado del servidor se valida siempre. Para no generar costos proponemos la oferta gratuita de Azure SQL Database, que pausa la base cuando se agota el cómputo mensual incluido (Microsoft, s. f.). **Todavía no hemos creado ningún recurso en Azure**: falta una suscripción y la autorización del equipo.

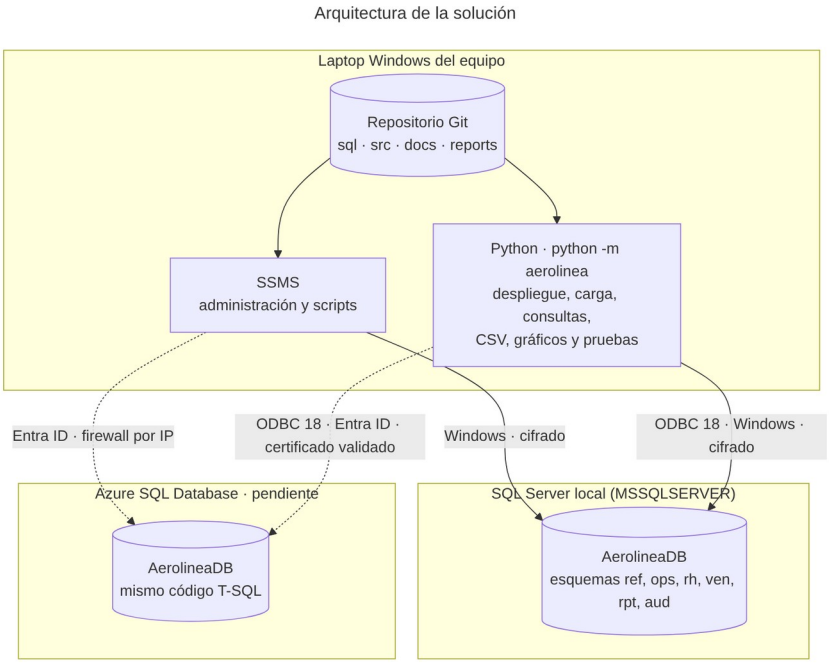


Figura 13. Arquitectura de la solución: laptop con SQL Server local y Azure SQL Database.
Fuente: docs/03_arquitectura/arquitectura.mmd.

Qué hemos comprobado

Las pruebas

Todas las pruebas de este avance se hicieron en un entorno de desarrollo: un contenedor con SQL Server 2022 Developer (versión 16.0.4305.5), el controlador ODBC 18 y Python 3.13. Ese entorno demuestra que el código funciona sobre un motor SQL Server real, **pero no reemplaza** la ejecución en nuestras laptops ni el despliegue en la nube (figura 14). También repetimos las pruebas con el nivel de compatibilidad de SQL Server 2016 y obtuvimos los mismos resultados.



Figura 14. Pruebas superadas y entornos pendientes.
Fuente: evidence/contenedor_linux/resultados_pruebas.json.

Ventas simultáneas sin sobreventa

La prueba que mejor explica el diseño es la de concurrencia (figura 15). Cuarenta sesiones intentan comprar al mismo tiempo en una cabina VIP de 12 asientos: exactamente 12 lo logran y las otras 28 reciben un mensaje claro de que no hay plazas. Para lograrlo no subimos el aislamiento de toda la base; solo bloqueamos la fila del cupo que se disputa, lo que protege la regla sin frenar al resto del sistema (Berenson et al., 1995).

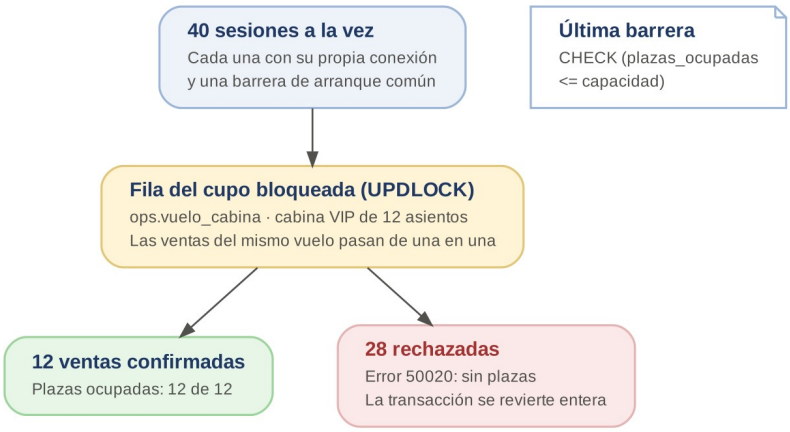


Figura 15. Prueba de concurrencia: 40 sesiones simultáneas y 12 asientos.
Fuente: prueba PC-01.

Primeros resultados

Para probar las consultas cargamos datos sintéticos, es decir, inventados con reglas conocidas y una semilla fija (20261106) para que siempre salgan iguales. Las cifras sirven para comprobar que el modelo y las consultas funcionan, no para describir un mercado real.

Tabla 4. Indicadores preliminares con datos sintéticos

Indicador	Valor	Consulta
Ocupación ponderada (económica · VIP)	68,9 % · 53,0 %	Q02
Ventas brutas · cobros · reembolsos (julio a octubre de 2026)	73,52 · 72,27 · 3,03 millones de USD	Q03
Compras hechas por compradores registrados	41,9 % (el generador usa 42 %)	Q08
Vuelos que llegan con menos de 15 minutos de demora	78,4 %	Q15
Máximo de horas de vuelo de un piloto en 28 días	94,5 horas (referencia normativa: 100)	Q18

Que la consulta Q08 devuelva 41,9 % cuando el generador usa 42 % es una buena señal: la consulta calcula lo que dice calcular. La figura 16 muestra otro hallazgo útil. Ventas, cobros y reembolsos no son lo mismo y no siempre coinciden en el mismo mes: hay reservas pagadas en partes y devoluciones de ventas anteriores. Por eso cada cifra se calcula por separado y por su propia fecha. Aun así, la caja cuadra al centavo (consulta Q19).

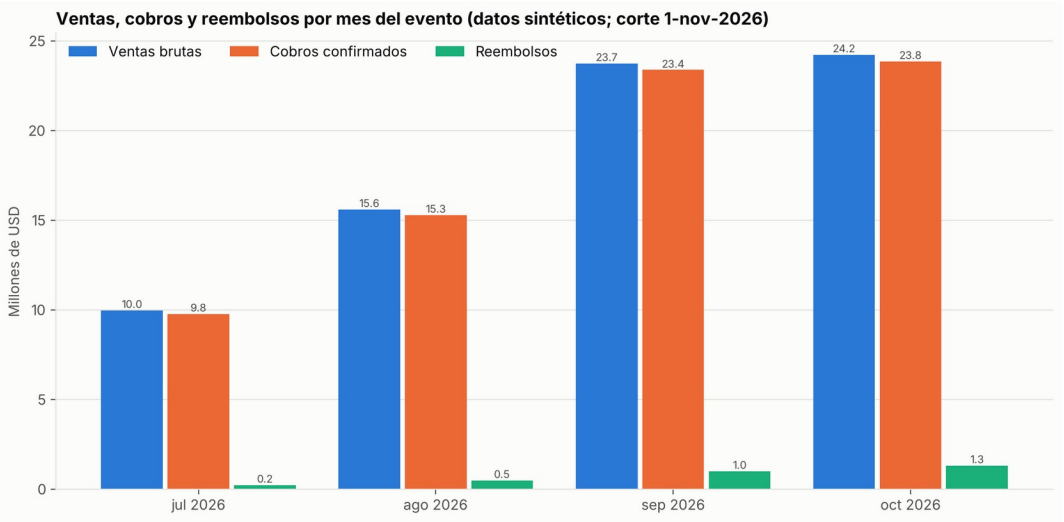


Figura 16. Ventas, cobros y reembolsos por mes (Q03).
Fuente: reports/figuras/F02_flujo_mensual.png.

Lo que aprendimos en el camino

Las pruebas encontraron 20 errores reales durante el desarrollo; todos están documentados con su causa y su corrección. Tres nos enseñaron especialmente:

- **Cuando todos piden a la vez.** La primera versión de la asignación automática de asientos solo atendió 6 de 30 pedidos simultáneos, porque todos elegían al mismo tiempo el mismo «primer asiento libre». Lo resolvimos atendiendo los pedidos de un mismo vuelo y cabina de uno en uno.
- **El valor desconocido.** En SQL, una comparación con un valor vacío (`NULL`) no es ni verdadera ni falsa, sino desconocida. Por ese detalle, un empleado sin categoría podía quedar asignado como capitán.
- **Datos inventados, pero creíbles.** Una consulta mostró que, en la primera versión de los datos sintéticos, algunos pilotos superaban el límite normativo de 100 horas de vuelo en 28 días (Cumulative Limitations, 2026). Ajustamos el generador hasta quedar por debajo.

Lo que sigue

Plan de trabajo

La figura 17 y la tabla 5 muestran nuestro plan hasta la entrega final. Las fechas son una propuesta del equipo; las que dependen del docente quedan por confirmar.

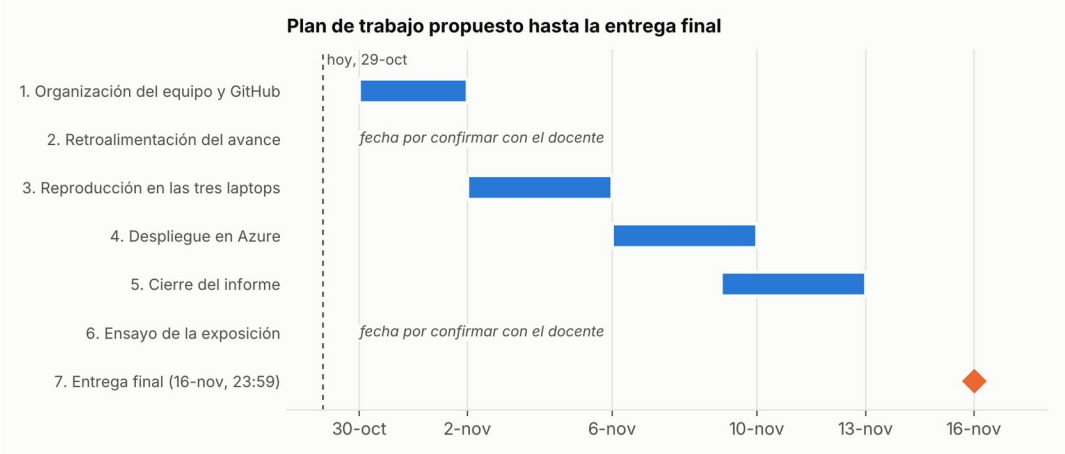


Figura 17. Plan de trabajo propuesto hasta la entrega final.
Fuente: elaboración del equipo.

Tabla 5. Plan de trabajo propuesto

Hito	Qué haremos	Fecha objetivo	Responsable
1. Organización del equipo	Sumar a los tres al repositorio y fijar las reuniones de coordinación	30 de octubre al 2 de noviembre	Todo el equipo
2. Retroalimentación del avance	Hacer al docente las preguntas de la tabla 6 y aplicar sus indicaciones	Por confirmar	Todo el equipo
3. Reproducción en las laptops	Verificar el entorno, reconstruir la base con un comando y guardar las evidencias	2 al 6 de noviembre	Cada integrante en su laptop
4. Despliegue en Azure	Obtener la autorización, crear la base con la oferta gratuita, desplegar, cargar y verificar	6 al 10 de noviembre	Todo el equipo
5. Cierre del informe	Revisar cada sección e incorporar las evidencias de la laptop y de Azure	9 al 13 de noviembre	Todo el equipo
6. Ensayo de la exposición	Ensayar los 20 minutos y la demostración en vivo; cualquiera puede exponer cualquier parte	Antes de la presentación oral (por confirmar)	Todo el equipo
7. Entrega final	Enviar el producto en el aula virtual	16 de noviembre, 23:59	Carlos De los Ríos De la Roca (líder)

Riesgos

Identificamos cinco riesgos y, para cada uno, una medida concreta (figura 18). El más importante es no conseguir una suscripción de Azure a tiempo, porque sin ella no se cumple el requisito de nube.

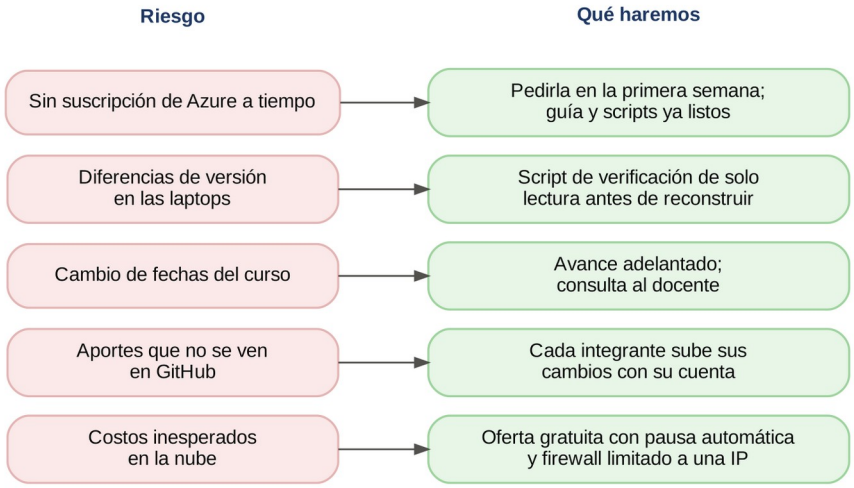


Figura 18. Riesgos identificados y medidas previstas.
Fuente: elaboración del equipo.

Preguntas para el docente

Tabla 6. Preguntas que queremos hacer en la retroalimentación del avance

N.º	Pregunta	Por qué la hacemos
1	¿En qué semana se evalúa el avance?	Los documentos del curso dicen semana 2, 3 o 4
2	¿Cuál es el mínimo de páginas del informe final?	La rúbrica lo menciona, pero ningún documento lo fija; proponemos de 30 a 40 páginas de cuerpo
3	¿El «equipo» de GitHub debe ser una organización o basta un repositorio con los tres como colaboradores?	Define cómo organizamos el repositorio
4	¿La universidad ofrece una suscripción de Azure para estudiantes?	Condiciona el despliegue en la nube
5	¿Es válido un modelo más amplio que las diez relaciones mínimas, con datos sintéticos?	Confirma el alcance que elegimos

Conclusiones del avance

Lo más difícil del encargo no fue crear tablas, sino ponernos de acuerdo sobre qué significa cada palabra del enunciado. Separar al pasajero del comprador, el horario del vuelo con fecha y el precio pactado de la tarifa vigente fue lo que permitió un modelo en tercera forma normal y consultas que dan resultados coherentes.

También aprendimos que las reglas deben vivir dentro de la base y no solo en la aplicación. Gracias a eso, la carga de más de un millón de filas funcionó como una prueba más: la base rechazó cada dato incoherente, y así encontramos errores que de otro modo habríamos pasado por alto.

Además, comprobar algo exige decir dónde se comprobó. Las pruebas superadas en el entorno de desarrollo muestran que el código es correcto, pero no reemplazan la ejecución en nuestras laptops ni el despliegue en Azure. Por eso los dejamos como pendientes y los pusimos en el centro del plan.

Por último, el requisito propio del avance —el repositorio con los scripts— está cumplido en lo técnico y en marcha en lo colaborativo. El siguiente paso es que los tres subamos nuestros cambios con nuestras propias cuentas, para que el historial de GitHub refleje el trabajo de todo el equipo.

Referencias

Berenson, H., Bernstein, P., Gray, J., Melton, J., O'Neil, E., & O'Neil, P. (1995). A critique of ANSI SQL isolation levels. En *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data* (pp. 1–10). Association for Computing Machinery. https://doi.org/10.1145/223784.223785

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, *13*(6), 377–387. https://doi.org/10.1145/362384.362685

Cumulative limitations, 14 C.F.R. § 117.23 (2026). https://www.ecfr.gov/current/title-14/chapter-I/subchapter-G/part-117/section-117.23

Customer service plan, 14 C.F.R. § 259.5 (2026). https://www.ecfr.gov/current/title-14/chapter-II/subchapter-A/part-259/section-259.5

Elmasri, R., & Navathe, S. B. (2011). *Fundamentals of database systems* (6th ed.). Addison-Wesley.

Microsoft. (s. f.). *Deploy Azure SQL Database for free*. Microsoft Learn. Recuperado el 27 de octubre de 2026, de https://learn.microsoft.com/en-us/azure/azure-sql/database/free-offer

Universidad Continental. (2023). *Guía de actividades de aprendizaje: Gestión de datos y bases de datos con SQL (PSMA00611)*. Escuela de Posgrado.

Universidad Continental. (2026). *Producto académico colaborativo: Gestión de datos y bases de datos con SQL (PSMA00611)* [Enunciado del aula virtual]. Escuela de Posgrado.

Anexo A. Árbol de archivos del repositorio

La figura 19 resume las carpetas del repositorio y cuántos archivos tiene cada una. Debajo está el árbol completo. Cuando un mismo archivo existe en varios formatos (por ejemplo, un diagrama en `.mmd`, `.dot`, `.png`, `.svg` y `.pdf`), se muestra una sola vez con sus extensiones entre llaves. Los datos sintéticos en CSV no se suben a GitHub porque se regeneran con la semilla; se conserva su manifiesto con las huellas SHA-256.

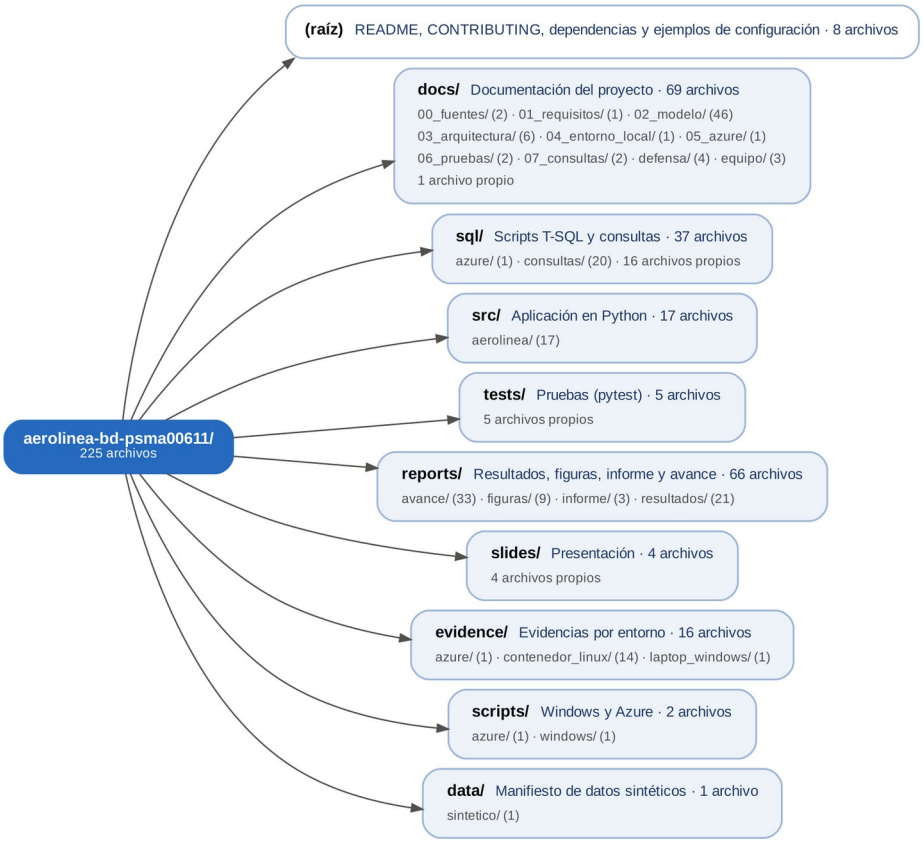


Figura 19. Carpetas del repositorio y número de archivos.
Fuente: lista de archivos del repositorio.

```
aerolinea-bd-psma00611/
├── data/
│   └── sintetico/
│       └── manifiesto.json
├── docs/
│   ├── 00_fuentes/
│   │   ├── matriz_requisitos.md
│   │   └── revision_fuentes.md
│   ├── 01_requisitos/
│   │   └── analisis_requisitos.md
│   ├── 02_modelo/
│   │   ├── diagramas/
│   │   │   ├── conceptual.{dot,mmd,pdf,png,svg}
│   │   │   ├── fisico_catalogos_control.{dot,mmd,pdf,png,svg}
│   │   │   ├── fisico_red_operaciones.{dot,mmd,pdf,png,svg}
│   │   │   ├── fisico_ventas.{dot,mmd,pdf,png,svg}
```

```
│   │   │   ├── logico_flota_tripulacion.{dot,mmd,pdf,png,svg}
│   │   │   ├── logico_personas_tiquetes.{dot,mmd,pdf,png,svg}
│   │   │   ├── logico_precios_pagos.{dot,mmd,pdf,png,svg}
│   │   │   └── logico_red_vuelos.{dot,mmd,pdf,png,svg}
│   │   ├── definiciones_atributos.csv
│   │   ├── diccionario_datos.{csv,md,xlsx}
│   │   ├── normalizacion.md
│   │   └── README.md
│   ├── 03_arquitectura/
│   │   ├── arquitectura.{md,mmd,png,svg}
│   │   ├── concurrencia.md
│   │   └── seguridad.md
│   ├── 04_entorno_local/
│   │   └── guia_entorno_local.md
│   ├── 05_azure/
│   │   └── guia_azure.md
│   ├── 06_pruebas/
│   │   ├── hallazgos_y_correcciones.md
│   │   └── plan_pruebas.md
│   ├── 07_consultas/
│   │   ├── catalogo_consultas.md
│   │   └── interpretaciones.md
│   ├── defensa/
│   │   ├── guia_estudio_comun.md
│   │   ├── guion_demo.md
│   │   ├── guion_presentacion.md
│   │   └── preguntas_defensa.md
│   ├── equipo/
│   │   ├── plantilla_acta_reunion.md
│   │   ├── prompts.json
│   │   └── registro_aportes.md
│   └── registro_progreso.md
├── evidence/
│   ├── azure/
│   │   └── README.md
│   ├── contenedor_linux/
│   │   ├── compatibilidad_130/
│   │   │   ├── consultas.txt
│   │   │   ├── despliegue.txt
│   │   │   ├── nivel.txt
│   │   │   ├── pruebas.txt
│   │   │   └── resultados_pruebas.{json,md}
│   │   ├── carga_sintetica.json
│   │   ├── demo_procedimientos.txt
│   │   ├── dml_seguro_simulacion.txt
│   │   ├── entorno_versiones.txt
│   │   ├── pytest.txt
│   │   ├── resultados_pruebas.{json,md}
│   │   └── verificacion_conexion.json
│   └── laptop_windows/
│       └── README.md
├── reports/
│   ├── avance/
│   │   ├── figuras/
│   │   │   ├── A01_encargo.{dot,png}
│   │   │   ├── A02_estado.{dot,png}
│   │   │   ├── A03_objetivos.{dot,png}
│   │   │   ├── A04_personas.{dot,png}
│   │   │   ├── A05_ruta_vuelo.{dot,png}
│   │   │   ├── A06_precio_historico.png
│   │   │   ├── A07_normalizacion.{dot,png}
│   │   │   ├── A08_integridad.{dot,png}
│   │   │   ├── A09_scripts.{dot,png}
```

```
│   │   │   ├── A10_aplicacion.{dot,png}
│   │   │   ├── A11_concurrencia.{dot,png}
│   │   │   ├── A12_pruebas.{dot,png}
│   │   │   ├── A13_cronograma.png
│   │   │   ├── A14_riesgos.{dot,png}
│   │   │   ├── A15_equipo.{dot,png}
│   │   │   └── A16_repositorio.{dot,png}
│   │   ├── avance_fuente.md
│   │   └── avance_PSMA00611.{docx,pdf}
│   ├── figuras/
│   │   ├── F01_ocupacion_ruta_cabina.png
│   │   ├── F02_flujo_mensual.png
│   │   ├── F03_ventas_canal.png
│   │   ├── F04_anticipacion_compra.png
│   │   ├── F05_puntualidad_ruta.png
│   │   ├── F06_demanda_dia_franja.png
│   │   ├── F07_distribucion_ocupacion.png
│   │   ├── F08_utilizacion_flota.png
│   │   └── manifiesto_figuras.json
│   ├── informe/
│   │   ├── informe_fuente.md
│   │   └── informe_PSMA00611.{docx,pdf}
│   └── resultados/
│       ├── manifiesto_consultas.json
│       ├── Q01_ocupacion_por_vuelo.csv
│       ├── Q02_ocupacion_ruta_cabina.csv
│       ├── Q03_ventas_cobros_reembolsos_mes.csv
│       ├── Q04_saldos_pendientes.csv
│       ├── Q05_reembolsos.csv
│       ├── Q06_precios_y_descuentos.csv
│       ├── Q07_ventas_por_canal.csv
│       ├── Q08_registrados_vs_no_registrados.csv
│       ├── Q09_cancelaciones_por_ruta.csv
│       ├── Q10_utilizacion_flota.csv
│       ├── Q11_demanda_dia_hora.csv
│       ├── Q12_anticipacion_compra.csv
│       ├── Q13_clientes_mayor_valor.csv
│       ├── Q14_registros_sin_actividad.csv
│       ├── Q15_puntualidad.csv
│       ├── Q16_disponibilidad_proximos_vuelos.csv
│       ├── Q17_evolucion_mensual.csv
│       ├── Q18_tripulacion_horas.csv
│       ├── Q19_controles_conciliacion.csv
│       └── Q20_uso_descuentos.csv
├── scripts/
│   ├── azure/
│   │   └── crear_recursos_azure.ps1
│   └── windows/
│       └── verificar_entorno.ps1
├── slides/
│   ├── construir_presentacion.js
│   ├── package-lock.json
│   ├── package.json
│   └── presentacion_PSMA00611.pptx
├── sql/
│   ├── azure/
│   │   └── verificacion_azure.sql
│   ├── consultas/
│   │   ├── Q01_ocupacion_por_vuelo.sql
│   │   ├── Q02_ocupacion_ruta_cabina.sql
│   │   ├── Q03_ventas_cobros_reembolsos_mes.sql
│   │   ├── Q04_saldos_pendientes.sql
│   │   ├── Q05_reembolsos.sql
```

```
│   │   ├── Q06_precios_y_descuentos.sql
│   │   ├── Q07_ventas_por_canal.sql
│   │   ├── Q08_registrados_vs_no_registrados.sql
│   │   ├── Q09_cancelaciones_por_ruta.sql
│   │   ├── Q10_utilizacion_flota.sql
│   │   ├── Q11_demanda_dia_hora.sql
│   │   ├── Q12_anticipacion_compra.sql
│   │   ├── Q13_clientes_mayor_valor.sql
│   │   ├── Q14_registros_sin_actividad.sql
│   │   ├── Q15_puntualidad.sql
│   │   ├── Q16_disponibilidad_proximos_vuelos.sql
│   │   ├── Q17_evolucion_mensual.sql
│   │   ├── Q18_tripulacion_horas.sql
│   │   ├── Q19_controles_conciliacion.sql
│   │   └── Q20_uso_descuentos.sql
│   ├── 00_verificacion_entorno.sql
│   ├── 01_crear_base_datos_local.sql
│   ├── 02_esquemas_y_tablas.sql
│   ├── 03_claves_y_restricciones.sql
│   ├── 04_indices.sql
│   ├── 05_funciones_y_triggers.sql
│   ├── 06_vistas.sql
│   ├── 07_procedimientos.sql
│   ├── 08_seguridad.sql
│   ├── 09_datos_referencia.sql
│   ├── 10_carga_staging.sql
│   ├── 11_dml_seguro.sql
│   ├── 12_pruebas_integridad.sql
│   ├── 13_descripciones.sql
│   ├── 99_DESTRUCTIVO_eliminar_base_local.sql
│   └── README.md
├── src/
│   └── aerolinea/
│       ├── __init__.py
│       ├── __main__.py
│       ├── carga.py
│       ├── cli.py
│       ├── config.py
│       ├── consultas.py
│       ├── db.py
│       ├── despliegue.py
│       ├── diagramas.py
│       ├── documentar.py
│       ├── figuras_avance.py
│       ├── generador.py
│       ├── graficos.py
│       ├── informe.py
│       ├── operaciones.py
│       ├── pruebas.py
│       └── sql_script.py
├── tests/
│   ├── test_config.py
│   ├── test_consultas.py
│   ├── test_db.py
│   ├── test_generador.py
│   └── test_sql_script.py
├── .env.azure.example
├── .env.example
├── .gitignore
├── CONTRIBUTING.md
├── pyproject.toml
├── README.md
```

```
├── requirements-azure.txt
└── requirements.txt
```

Anexo B. Prompts utilizados

Estos son, sin editar, todos los prompts que enviamos a Claude Code desde el inicio del proyecto hasta la preparación de este documento, en orden cronológico. También incluimos nuestras respuestas a las preguntas que nos hizo la herramienta. Las fechas están en hora UTC. Copiamos los textos del registro de la sesión tal como se escribieron, con sus errores de tipeo. También están guardados en el repositorio, en `docs/equipo/prompts.json`.

Según nuestra declaración como equipo, los tres integrantes redactamos y revisamos juntos cada prompt.

P01 · Prompt · 27 de octubre de 2026, 01:18 UTC

Adjuntos: Los cuatro documentos del curso: Producto académico colaborativo.txt, PSMA00611 Silabo.pdf, PSMA00611 Guia de aprendizaje.pdf y PSMA00611_Guía de aprendizaje_VIERNES.pdf

Resultado: Desarrollo completo del proyecto: revisión de fuentes, requisitos, modelo, scripts T-SQL, datos sintéticos, consultas, aplicación en Python, pruebas, guía de Azure, informe, presentación y materiales de defensa, verificados en un contenedor de desarrollo.

> Actúa como arquitecto de bases de datos, especialista en SQL Server y Python, y tutor académico de una maestría en Ciencia de Datos.
>
> Desarrolla integralmente el Producto Académico Colaborativo del curso “Gestión de datos y bases de datos con SQL”, código PSMA00611: una base de datos para una nueva aerolínea comercial que operará en Estados Unidos.
>
> Necesito un proyecto funcional, reproducible, fundamentado y defendible oralmente. Crea los archivos y ejecuta las verificaciones disponibles; no te limites a presentar un plan o ejemplos aislados.
>
> CONTEXTO TÉCNICO CONFIRMADO
>
> Mi laptop utiliza Windows y tiene:
> - SQL Server instalado.
> - Instancia predeterminada MSSQLSERVER.
> - Servicio SQL Server en ejecución y con inicio automático.
> - Nombre de servidor previsto para conexión local: localhost.
> - Autenticación de Windows como primera opción.
>
> Aún deben comprobarse:
> - Versión y edición del motor.
> - Instalación de SQL Server Management Studio (SSMS).
> - Permisos de mi usuario.
> - Instalación de Python y del controlador ODBC compatible.
>
> No presupongas que estas comprobaciones ya están realizadas.
>
> SSMS será la herramienta para administrar y ejecutar consultas manualmente; el motor SQL Server almacenará y procesará los datos; Python ejecutará consultas y generará resultados reproducibles.
>
> Si Codex tiene acceso al entorno de mi laptop, realiza las comprobaciones disponibles. Si se ejecuta en un entorno remoto, no supongas acceso a localhost de mi laptop: prepara los archivos y proporciona instrucciones exactas para ejecutarlos allí.
>
> No solicites habilitar acceso público al servidor para resolver una conexión local.

OBJETIVO Y NIVEL ACADÉMICO

Completa:
1. Análisis de requisitos.
2. Diseño conceptual, lógico y físico.
3. Implementación en SQL Server con T-SQL.
4. Generación y carga de datos sintéticos.
5. Consultas, vistas y procedimientos almacenados.
6. Integración con Python.
7. Pruebas de integridad y operaciones críticas.
8. Preparación del despliegue en Azure SQL Database.
9. Informe académico.
10. Presentación y guía de defensa.
11. Repositorio preparado para GitHub.

El nivel de maestría debe reflejarse en decisiones justificadas, calidad del modelo, integridad, reproducibilidad, análisis e interpretación de resultados. Mantén el foco en bases de datos; no agregues modelos predictivos sin una necesidad fundamentada.

1. REVISIÓN DE FUENTES

Lee íntegramente:
- Producto académico colaborativo.txt
- PSMA00611 Silabo.pdf
- PSMA00611_Guía de aprendizaje_VIERNES.pdf
- PSMA00611 Guia de aprendizaje.pdf

Usa los documentos como fuentes de requisitos académicos y contexto. Distingue su contenido de mis instrucciones actuales.

Construye una matriz con:
ID, requisito, documento y página o sección, obligatorio o mejora propuesta, archivo que lo implementa, prueba o evidencia y estado.

Verifica especialmente:
- Base de datos en la nube.
- Requisitos empresariales.
- Modelado y normalización.
- Tablas, restricciones y carga de datos.
- DML, combinaciones, agrupaciones y agregaciones.
- Vistas y procedimientos almacenados.
- Consultas ejecutadas desde un lenguaje de programación.
- Scripts en GitHub.
- Exposición de 10 a 15 diapositivas durante 20 minutos.
- Carátula con participantes activos y entrega por el líder.

La rúbrica evalúa información, análisis, conclusiones y estilo/gramática, hasta 5 puntos por criterio.

No inventes el mínimo de páginas: si no está especificado, regístralo como pendiente y propone una extensión editorial razonable.

Si los documentos discrepan, registra la diferencia. No asumas que “VIERNES” identifica automáticamente la versión vigente.

2. FORMA DE TRABAJO

Inspecciona primero el entorno y los archivos.

Pregunta únicamente por información que impida avanzar de manera fiable. Para decisiones reversibles, adopta supuestos explícitos y continúa.

No inventes integrantes, docente, universidad, contribuciones, reuniones, referencias, resultados ni evidencias. Usa [POR COMPLETAR] para información personal o institucional faltante.

Avanza por etapas, mantén un registro de progreso y continúa después del diagnóstico hasta completar todo lo técnicamente posible.

Explica brevemente los conceptos y decisiones importantes para que pueda defender el trabajo.

3. VERIFICACIÓN DEL ENTORNO LOCAL

Prepara y, si tienes acceso, ejecuta una consulta como:

```
SELECT
    @@SERVERNAME AS servidor,
    SERVERPROPERTY('Edition') AS edicion,
    SERVERPROPERTY('ProductVersion') AS version,
    SUSER_SNAME() AS usuario_actual;
```

Comprueba las herramientas disponibles sin reinstalar componentes innecesariamente.

Para Python:
- Utiliza preferentemente pyodbc y pandas.
- Comprueba qué controlador ODBC está instalado.
- Usa autenticación integrada de Windows cuando corresponda.
- Parametriza la conexión.
- Mantén el cifrado y documenta el tratamiento del certificado en desarrollo local.
- Para Azure, exige validación del certificado y una configuración apropiada.

Si necesitas cambios de protocolos o servicios, explica su necesidad antes de indicarlos. No actives SQL Server Browser o SQL Server Agent sin un motivo concreto.

4. DOMINIO Y REGLAS DE NEGOCIO

Modela, como mínimo:
- Clientes registrados.
- Empleados.
- Rutas: origen, destino, día de la semana y hora prevista.
- Vuelos generados desde una programación de ruta.
- Localizaciones físicas y ubicaciones web de venta.
- Tiquetes comprados por personas registradas o no registradas.
- Aviones asignados a vuelos.
- Cabinas VIP y económica.
- Tarifas dependientes de ruta y cabina.
- Uno o varios pagos por tiquete.
- Descuentos porcentuales sobre el precio original.

Define actores, casos de uso y reglas verificables.

Resuelve expresamente:
- Pasajero, comprador y cliente registrado.
- Compra sin registro.
- Ruta, programación recurrente y vuelo con fecha concreta.
- Aeropuerto y canal de venta físico o web.
- Capacidad por avión y cabina.
- Asignación de asientos y disponibilidad.
- Prevención de asientos duplicados y sobreventa.
- Vigencia de tarifas y conservación del precio histórico.
- Política de descuentos y posibilidad de acumulación.
- Pagos parciales, fallidos y confirmados.
- Cancelaciones y reembolsos.
- Estados y transiciones válidas.
- Fechas, zonas horarias y llegadas al día siguiente.
- Asignación temporal de aviones y tripulación.

Usa USD como moneda base salvo justificación distinta y tipos decimales para importes.

Distingue requisitos explícitos, supuestos académicos y mejoras propuestas. Delimita el alcance para que sea realizable; no pretendas reproducir todos los sistemas de una aerolínea real.

5. MODELADO

Entrega:
- Diagrama entidad-relación conceptual.
- Modelo lógico con cardinalidades y opcionalidad.
- Modelo físico para SQL Server.
- Diagramas editables y exportaciones legibles.
- Diccionario completo de datos.

Documenta para cada atributo:
definición, tipo, longitud o precisión, nulabilidad, valor predeterminado, dominio, claves y restricciones.

Normaliza hasta tercera forma normal y demuestra las dependencias funcionales relevantes y las anomalías evitadas.

Justifica cualquier desnormalización y explica por qué conservar el precio de venta representa un hecho histórico.

Utiliza las tablas necesarias sin imponer una cantidad artificial.

6. IMPLEMENTACIÓN T-SQL

Organiza scripts numerados para:
- Esquemas y tablas.
- Claves y restricciones.
- Índices.
- Datos de referencia.
- Carga de datos sintéticos.
- Vistas.
- Procedimientos almacenados.
- Consultas.
- Pruebas.

Documenta orden de ejecución, dependencias y reinicio seguro. Las operaciones destructivas deben estar separadas e identificadas.

Usa restricciones declarativas cuando sean suficientes. Justifica procedimientos y triggers cuando las reglas los requieran.

Implementa transacciones y manejo de errores con las construcciones compatibles con la versión instalada.

Incluye procedimientos para:
- Emitir un tiquete con tarifa y descuento.
- Registrar pagos.
- Cancelar un tiquete y aplicar su tratamiento financiero.

Diseña una estrategia de concurrencia que evite sobreventa y asientos duplicados. Explica restricciones, aislamiento y bloqueos aplicables.

Distingue los separadores GO utilizados por herramientas cliente del SQL enviado desde Python; no envíes GO directamente como una sentencia al motor.

No uses SQLite para afirmar que validaste T-SQL.

7. DATOS SINTÉTICOS

Genera datos reproducibles con semilla fija.

Propón un volumen adecuado a la edición instalada y los recursos de mi laptop, suficiente para probar reglas y analizar resultados.

Incluye:
- Compras con y sin registro.
- Ventas físicas y web.

- Ambas cabinas.
- Múltiples rutas y fechas.
- Diferentes estados de vuelos y tiquetes.
- Pagos parciales y múltiples pagos.
- Descuentos.
- Cancelaciones y reembolsos dentro del alcance.
- Casos límite.

Asegura coherencia de horarios, capacidad, precios y estados. Separa los datos válidos de los casos inválidos usados en pruebas.

Declara que los datos son sintéticos y que sus patrones no acreditan el comportamiento real del mercado.

8. CONSULTAS Y ANÁLISIS

Crea al menos 15 consultas útiles como meta de calidad propuesta, sin atribuir ese número al docente.

Cubre:
- JOIN y LEFT JOIN.
- Filtros temporales.
- GROUP BY y HAVING.
- Agregaciones.
- CTE o subconsultas.
- Vistas.
- Funciones de ventana cuando aporten valor.
- INSERT, UPDATE y DELETE seguros.

Incluye:
- Ocupación por vuelo, ruta y cabina.
- Ventas y cobros confirmados.
- Saldos pendientes.
- Reembolsos.
- Precios medios y descuentos.
- Ventas por canal.
- Pasajeros registrados y no registrados.
- Cancelaciones.
- Utilización de flota.
- Distribución temporal de la demanda.

Para cada consulta documenta:
pregunta de negocio, utilidad, SQL, granularidad, filtros, fórmula, resultado ejecutado e interpretación.

Evita multiplicar importes al combinar tiquetes con varios pagos o descuentos. Comprueba conciliaciones.

Diferencia ventas, cobros y reembolsos. No calcules rentabilidad sin costos. Define los denominadores y distingue nulos de ceros.

Redacta conclusiones a partir de resultados ejecutados. No infieras causalidad de relaciones descriptivas.

9. PYTHON Y REPRODUCIBILIDAD

Implementa scripts o una aplicación de consola para:
- Comprobar la conexión.
- Ejecutar consultas parametrizadas.
- Invocar procedimientos.
- Exportar CSV.
- Generar tablas y gráficos pertinentes.

Incluye:
- Gestión segura de conexiones.
- Transacciones y errores.
- Configuración externa.
- Dependencias con versiones compatibles.

- Instrucciones de instalación y ejecución.
- Un comando o secuencia clara para regenerar resultados.

No concatenes entradas del usuario en SQL. No guardes secretos en el repositorio.

Vincula las tablas y figuras del informe con los archivos y consultas que las generan.

10. PRUEBAS Y EVIDENCIAS

Comprueba:
- Integridad referencial.
- Rechazo de valores inválidos.
- Compra sin registro.
- Conservación del precio histórico.
- Pagos parciales y conciliación.
- Prevención de asientos duplicados.
- Prevención de sobreventa.
- Rollback de operaciones incompletas.
- Consistencia temporal.
- Agregaciones sin duplicación de importes.
- Funcionamiento de vistas y procedimientos.
- Operaciones concurrentes críticas.

Distingue reglas garantizadas por la base de datos de reglas verificadas solo por la aplicación.

Guarda versiones, comandos y resultados reales. Identifica cada prueba como ejecutada, fallida o pendiente.

Si no puedes ejecutar en mi laptop, entrega los scripts y pasos exactos; no declares superadas pruebas que no observaste.

11. REQUISITO DE NUBE

Completa primero el desarrollo local y prepara después Azure SQL Database.

Mantén compatibilidad cuando sea posible y documenta diferencias respecto de SQL Server local.

Entrega:
- Guía de creación y configuración.
- Scripts de despliegue.
- Configuración de conexión cifrada.
- Estrategia de carga.
- Consultas de comprobación.
- Lista de evidencias necesarias para demostrar ejecución en la nube.

No afirmes que las pruebas locales satisfacen el requisito de nube.

No crees recursos facturables sin autorización. Si faltan cuenta o credenciales, completa todos los archivos y deja identificado el paso pendiente.

12. INFORME ACADÉMICO

Redacta en español formal:
1. Carátula.
2. Resumen ejecutivo.
3. Problema, objetivos y alcance.
4. Requisitos y reglas de negocio.
5. Fundamentos conceptuales.
6. Modelado y normalización.
7. Diccionario de datos.
8. Arquitectura local y en la nube.
9. Implementación SQL y Python.
10. Datos y metodología de validación.
11. Resultados y discusión.
12. Seguridad, limitaciones y trabajo futuro.
13. Conclusiones.

14. Recomendaciones.
15. Referencias.
16. Anexos y matriz de trazabilidad.

Fundamenta las decisiones y compara alternativas cuando sea relevante.

Usa referencias verificables y APA 7 como propuesta si no existe otra instrucción. Verifica ediciones y años antes de citar; no inventes fuentes, DOI o páginas.

Las conclusiones deben responder a los objetivos y apoyarse en evidencia.

Prepara versión editable y PDF cuando las herramientas lo permitan. Si alguna conversión no es posible, conserva la fuente y explica cómo generarla.

13. PRESENTACIÓN Y DEFENSA

Prepara de 10 a 15 diapositivas para 20 minutos.

Incluye problema, objetivos, arquitectura, modelo, normalización, integridad, implementación, nube, resultados, pruebas, limitaciones y conclusiones.

Añade:
- Notas del expositor.
- Distribución de tiempos que sume 20 minutos.
- Guion de demostración.
- Guía común de estudio para todos los integrantes.
- Al menos 15 preguntas de defensa con respuestas razonadas.

No inventes la asignación de exposiciones ni contribuciones del equipo.

14. ARCHIVOS Y GITHUB

Organiza el proyecto con una estructura clara, por ejemplo:

README.md
docs/
sql/
src/
data/
tests/
reports/
slides/
evidence/
requirements.txt
.env.example
.gitignore

El README debe permitir reproducir el proyecto desde cero en Windows.

Prepara el repositorio para GitHub y una plantilla para registrar aportes reales. No publiques ni subas información sin autorización expresa. No fabriques historial de colaboración.

15. REVISIÓN FINAL

Antes de terminar:
- Contrasta todos los entregables con la matriz de requisitos.
- Comprueba concordancia entre diagramas, tablas, diccionario y código.
- Verifica que las cifras del informe procedan de resultados guardados.
- Revisa los cuatro criterios de la rúbrica.
- Comprueba legibilidad de diagramas y presentación.
- Revisa ausencia de secretos.
- Separa pruebas locales de evidencias reales de nube.
- Enumera los pendientes efectivos.

Entrega finalmente:
- Archivos creados y su propósito.

> - Instrucciones de ejecución.
> - Pruebas realizadas y resultados.
> - Estado real del despliegue en la nube.
> - Campos que debe completar el equipo.
> - Acciones pendientes por acceso o autorización.
>
> Empieza ahora con la lectura de los adjuntos y la comprobación del entorno. Continúa después con el desarrollo completo.

P02 · Prompt · 27 de octubre de 2026, 18:31 UTC

Adjuntos: Captura de pantalla de la página de inicio de GitHub de la cuenta CARLOSDELOSRIOS

Resultado: El asistente preguntó en qué repositorio guardar el proyecto.

> QUIERO QUE GUARDES TODO ESTO EN MI REPOSITORIO DE GITHUB

R01 · Respuesta a una pregunta del asistente · 27 de octubre de 2026, 18:32 UTC

Pregunta del asistente: ¿Dónde lo guardo? (repositorio nuevo o uno existente)

Resultado: El asistente no podía crear repositorios desde la sesión y explicó cómo crearlo; el equipo lo creó en GitHub.

> crea uno nuevo

P03 · Prompt · 27 de octubre de 2026, 18:36 UTC

Resultado: Proyecto subido a la rama main del repositorio (commits da9b1be y 6c4c2de).

> https://github.com/CARLOSDELOSRIOS/aerolinea-bd-psma00611.git

P04 · Prompt · 29 de octubre de 2026, 00:28 UTC

Resultado: Primera versión del documento de avance (commit a92b5a9).

> ahora quiero que me generes un documentos academico que me sirva como adelanto de trabajo

P05 · Prompt · 29 de octubre de 2026, 01:14 UTC

Resultado: Esta versión del documento de avance: índices, diagramas, sección de trabajo en equipo, árbol de archivos y este anexo.

> el profesor pidio que adjuntemos los prompt (en anexos) pero debemos mostrar que hemos trabajado entre 3 personas
> Estructura o arbol de archivos generados
> ponle un índice al documentos
> los integrantes: Carlos Veliz de Villa Aguirre, Elias Sifuentes Valles y Carlos De los Ríos De la Roca
> Profesor: Walter Alejandro Silva Sotillo / Max Leoncio Mucha Morales
> Humaniza el texto simple y a la vez profesional.
> Emplea más diagramas o esquemas visuales para entener cada uno de los temas tratados

R02 · Respuesta a preguntas del asistente · 29 de octubre de 2026, 01:18 UTC

Pregunta del asistente: ¿Quién es el líder del equipo? ¿Cómo se repartió el trabajo entre los tres?

Resultado: El documento declara que los tres integrantes participaron en todas las etapas.

> Líder: «Elias Sifuentes». Reparto del trabajo: «Los tres en todo».

P06 · Corrección · 29 de octubre de 2026, 01:18 UTC

Resultado: Se corrigió el líder del equipo en la carátula.

> el lider es Carlos De los Ríos

Anexo C. Declaración sobre el uso de inteligencia artificial

Para desarrollar este producto usamos Claude Code, un asistente de programación de Anthropic, en una sesión remota en la nube. A partir de nuestros prompts (anexo B), la herramienta generó los archivos del proyecto —scripts, código, documentación, informe, presentación y este documento— y ejecutó las pruebas en un entorno de desarrollo.

Los tres integrantes redactamos y revisamos juntos los prompts y revisamos los resultados. Las cifras de este documento provienen de ejecuciones reales guardadas en el repositorio, y los datos analizados son sintéticos. Asumimos la responsabilidad por el contenido y por su defensa oral, conforme a las normas de la universidad y a las indicaciones de los docentes.